

Funzioni

Funzioni

Come organizzare il codice PHP in blocchi riutilizzabili.

Cosa è una funzione

Una funzione è un blocco di codice con un nome. La scrivi una volta e puoi usarla più volte.

E come una ricetta: invece di riscrivere tutti i passaggi, richiami la ricetta quando serve.

Dichiarare una funzione

```
<?php
function saluta() {
    echo "Ciao!";
}
```

Questo codice crea la funzione, ma non la esegue ancora.

Chiamare una funzione

Per eseguirla, scrivi il suo nome seguito da parentesi.

```
<?php
function saluta() {
    echo "Ciao!";
}

saluta();
```

Output:

Ciao!

Riutilizzare codice

Una funzione evita ripetizioni.

```
<?php
function mostraSeparatore() {
    echo "---\n";
}

mostraSeparatore();
echo "Prodotti\n";
mostraSeparatore();
echo "Totale\n";
mostraSeparatore();
```

Se un giorno vuoi cambiare il separatore, modifichi una sola funzione.

Nomi chiari

Il nome deve dire cosa fa la funzione.

```
<?php
function calcolaTotale() {
    echo 10 + 5;
}
```

`calcolaTotale` è più utile di `faiCosa` .

Suggerimento: una funzione dovrebbe fare una cosa principale. Se fa troppe cose, diventa difficile da capire e da riusare.

Una funzione con un compito chiaro

Immagina di dover mostrare piu volte il prezzo con la parola `euro` .

```
<?php
function mostraPrezzo($prezzo) {
    echo $prezzo . " euro";
}

mostraPrezzo(12);
```

La funzione non conosce tutti i prodotti del programma. Si occupa solo di mostrare un prezzo. Questo la rende facile da capire.

Prima senza funzione, poi con funzione

All'inizio potresti scrivere:

```
<?php
echo "Ciao Luca\n";
echo "Ciao Sara\n";
```

Con una funzione elimini la ripetizione:

```
<?php
function saluta($nome) {
    echo "Ciao $nome\n";
}

saluta("Luca");
saluta("Sara");
```

Il vantaggio non e scrivere meno righe a tutti i costi. Il vantaggio e dare un nome a un'azione.

Errore comune: chiamare la funzione con un nome diverso

```
<?php
function saluta() {
    echo "Ciao";
}

saluto();
```

`saluta` e `saluto` sono due nomi diversi. PHP cercherà una funzione che non esiste.

Parametri e valori di ritorno

Come passare dati a una funzione e ricevere un risultato.

Parametri e argomenti

Una funzione può ricevere dati dall'esterno. I nomi scritti nella definizione si chiamano **parametri**.

```
<?php
function saluta($nome) {
    echo "Ciao $nome";
}

saluta("Luca");
```

`$nome` è il parametro. `"Luca"` è l'argomento passato quando chiami la funzione.

Piu parametri

```
<?php
function somma($a, $b) {
    echo $a + $b;
}

somma(3, 4);
```

Output:

7

return

return restituisce un valore a chi ha chiamato la funzione.

```
<?php
function somma($a, $b) {
    return $a + $b;
}

$totale = somma(3, 4);
echo $totale;
```

La funzione non stampa direttamente. Calcola e consegna il risultato.

Valori predefiniti

Puoi dare un valore di default a un parametro.

```
<?php
function saluta($nome = "ospite") {
    return "Ciao $nome";
}

echo saluta();
```

Output:

```
Ciao ospite
```

Type declarations di base

Puoi indicare che tipo di dato ti aspetti.

```
<?php
function calcolaIva(float $prezzo): float {
    return $prezzo * 0.22;
}
```

`float $prezzo` dice che il parametro deve essere un numero decimale. `: float` dice che la funzione restituisce un numero decimale.

Queste indicazioni rendono gli errori più facili da trovare.

echo o return?

Usa `echo` quando vuoi mostrare qualcosa subito. Usa `return` quando vuoi consegnare un valore al resto del programma.

```
<?php
function creaSaluto($nome) {
    return "Ciao $nome";
}

$messaggio = creaSaluto("Luca");
echo $messaggio;
```

Questa funzione è più flessibile: puoi stampare il messaggio, salvarlo, mandarlo in una email o combinarlo con altro testo.

Parametri con nomi chiari

I parametri sono variabili locali della funzione. Scegli nomi che spiegano il loro ruolo.

```
<?php
function calcolaTotale($prezzo, $quantita) {
    return $prezzo * $quantita;
}
```

`$prezzo` e `$quantita` sono più chiari di `$a` e `$b`, tranne negli esempi matematici molto piccoli.

Errore comune: dimenticare di usare il valore di ritorno

```
<?php
function somma($a, $b) {
    return $a + $b;
}

somma(2, 3);
```

La funzione calcola `5`, ma il valore viene perso. Se ti serve, salvalo o stampalo:

```
<?php
$totale = somma(2, 3);
echo $totale;
```

Scope delle variabili

Dove esistono le variabili e come evitare confusione tra funzioni e codice esterno.

Cosa significa scope

Lo **scope** è la zona in cui una variabile esiste ed è leggibile.

Una variabile creata dentro una funzione non è automaticamente disponibile fuori. Una variabile creata fuori non entra automaticamente dentro.

Variabili locali

```
<?php
function saluta() {
    $nome = "Luca";
    echo $nome;
}

saluta();
```

`$nome` esiste solo dentro la funzione `saluta` .

Questo codice invece non funziona come potresti aspettarti:


```
<?php
function preparaNome() {
    $nome = "Luca";
}

preparaNome();
echo $nome;
```

Fuori dalla funzione, `$nome` non esiste.

Parametri

Il modo più pulito per portare dati dentro una funzione e usare parametri.

```
<?php
function saluta($nome) {
    echo "Ciao $nome";
}

saluta("Sara");
```

Valori di ritorno

Il modo più pulito per portare dati fuori da una funzione e usare `return`.

```
<?php
function creaMessaggio($nome) {
    return "Ciao $nome";
}

$messaggio = creaMessaggio("Marta");
echo $messaggio;
```

Variabili globali

PHP permette di usare `global`, ma è meglio limitarlo.

```
<?php
$iva = 22;

function mostraIva() {
    global $iva;
    echo $iva;
}
```

Il codice funziona, ma la funzione dipende da qualcosa che sta fuori. Con parametri e `return`, il collegamento è più chiaro.

Perché lo scope aiuta

Lo scope evita che una variabile venga modificata per sbaglio da una parte lontana del programma. Una funzione con parametri chiari è più prevedibile.

```
<?php
function applicaSconto($prezzo, $sconto) {
    return $prezzo - $sconto;
}

$totale = applicaSconto(30, 5);
echo $totale;
```

La funzione lavora solo con i valori che riceve. Non deve cercare variabili nascoste nel resto del file.

Stesso nome, scope diverso

Puoi avere variabili con lo stesso nome in scope diversi.

```
<?php
$nome = "Luca";

function saluta() {
    $nome = "Sara";
    echo $nome;
}

saluta();
echo $nome;
```

Dentro la funzione `$nome` vale `Sara` . Fuori vale ancora `Luca` .

Regola pratica

Quando scrivi una funzione, prova a farle ricevere tutto cio che le serve tramite parametri. Se deve produrre un risultato, usa `return` . Tieni `global` come ultima scelta.